

Remote Procedure Call (RPC)

Concepts, Technologies, Failure Modes, and Comprehensive Comparative Analysis

Taha Majlesi
Student ID: 810101504
t.majlesi@ut.ac.ir

Spring 2026
University of Tehran

Contents

1	Introduction	5
2	Definition of RPC	5
2.1	Why “Remote” and “Procedure”?	5
2.2	Analogy: A Telephone Call	5
2.3	Historical Background	5
2.4	Key Characteristics	5
3	Core Idea: Calling a Remote Function	6
3.1	Detailed Call Flow (Step by Step)	6
3.2	Flow Diagram (Text Representation)	7
4	Difference Between Local and Remote Invocation	7
4.1	Why the Differences Matter in Practice	8
5	Role of Client Stub and Server Stub	8
5.1	Client Stub (Proxy)	8
5.2	Server Stub (Skeleton)	9
5.3	Code Generation and Tooling	9
6	Serialization and Deserialization	9
6.1	Deep Dive into Serialization Formats	10
6.2	Schema Evolution - Why It Matters	10
6.2.1	Rules for Safe Evolution (Protobuf Example)	10
6.2.2	Example of Evolution	10
6.3	Performance Considerations	11
7	Interface Definition Language (IDL)	11
7.1	Why IDL is Important	11
7.2	Examples of IDLs	11
7.2.1	Protocol Buffers (‘.proto’)	11
7.2.2	Thrift IDL	12
7.2.3	WSDL (XML)	12
7.2.4	OpenAPI (Swagger)	12
7.3	Example: Generating Stubs from ‘auth.proto’	12
7.4	Best Practices for IDL Design	13
7.5	IDL vs. No IDL (e.g., JSON-RPC)	13

8	Common RPC Errors - Exhaustive Taxonomy	13
8.1	1. Network Errors	14
8.2	2. Protocol Errors	14
8.3	3. Timeout Errors	14
8.4	4. Server Failures	15
8.5	5. Semantic Errors	15
8.6	6. Security Errors	15
9	Comparison of RPC and REST	16
9.1	Detailed Analysis of Each Criterion	16
9.1.1	Abstraction	16
9.1.2	HTTP Methods	16
9.1.3	Idempotency	16
9.1.4	Caching	16
9.1.5	Error Handling	17
9.1.6	Tooling	17
9.1.7	Streaming	17
9.1.8	Performance	17
9.1.9	Discoverability	17
9.1.10	Statefulness	17
9.1.11	Human-readability	17
9.2	When to Choose Which?	17
10	Comparison of Popular RPC Technologies	18
10.1	gRPC - Deep Dive	18
10.1.1	Protocol Buffers (Protobuf) - The Engine	18
10.1.2	HTTP/2 - The Transport	18
10.1.3	Performance and Benchmarks	18
10.1.4	Built-in Production Features	19
10.1.5	Limitations of gRPC	19
10.2	JSON-RPC - Deep Dive	19
10.2.1	Message Format	19
10.2.2	Key Features	19
10.2.3	Use Cases	19
10.2.4	Limitations	19
10.3	XML-RPC - Deep Dive	20
10.3.1	Message Structure	20
10.3.2	Why It Still Exists	20
10.3.3	Limitations	20
10.4	Additional Technologies - Brief Overview	20
10.4.1	Apache Thrift	20
10.4.2	Cap'n Proto	20
10.4.3	SOAP (Simple Object Access Protocol)	20
10.4.4	Java RMI (Remote Method Invocation)	21
10.5	Practical Selection Guidelines	21
11	Use Case Scenarios - Detailed Recommendations	22
11.1	High-performance Microservices	22
11.2	Browser-to-server Public API	22
11.3	Real-time Streaming	23
11.4	Simple Internal Script Calling a Remote Function	23
11.5	Legacy Enterprise Integration	23
11.6	Mobile Client Backend	23
11.7	IoT Devices (Constrained Resources)	24
11.8	Cross-language Systems with Custom Transport	24
11.9	Zero-copy, Ultra-high Performance	24

12 Advanced Topics in RPC	24
12.1 Idempotency and Exactly-Once Semantics	24
12.1.1 Definition and Importance	24
12.1.2 Examples of Idempotent Operations	24
12.1.3 Non-Idempotent Operations	25
12.1.4 Mitigation Strategies	25
12.1.5 Exactly-Once Semantics	25
12.2 Timeouts and Deadlines	25
12.2.1 Why Deadlines are Critical	25
12.2.2 How gRPC Implements Deadlines	25
12.2.3 Choosing a Deadline	25
12.3 Security in RPC	25
12.3.1 Encryption (TLS/SSL)	25
12.3.2 Authentication and Authorisation	26
12.3.3 JSON-RPC and XML-RPC Security	26
12.3.4 Common Pitfalls	26
12.4 Load Balancing and Service Discovery	26
12.4.1 Why Load Balancing is Needed	26
12.4.2 Client-Side vs Proxy-Side Load Balancing	26
12.4.3 Service Discovery	26
12.5 Protocol Evolution and Backward Compatibility	26
12.5.1 Why Schema Evolution Matters	26
12.5.2 Protobuf Evolution Rules	27
12.5.3 JSON-RPC and XML-RPC Evolution	27
12.6 Observability and Distributed Tracing	27
12.6.1 The Three Pillars of Observability	27
12.6.2 Correlation IDs	27
12.6.3 Tracing with gRPC	27
12.6.4 Manual Implementation for JSON-RPC / XML-RPC	27
12.7 Conclusion of Advanced Topics	28
13 Answers to Analytical Questions	28
13.1 1. Why does RPC make a remote service call appear like a local function call?	28
13.2 2. Why can this similarity be dangerous or misleading?	28
13.3 3. What errors can occur in remote invocations that do not exist in local invocations?	29
13.3.1 3.1 Network Errors	29
13.3.2 3.2 Protocol and Marshaling Errors	30
13.3.3 3.3 Timeout Errors	30
13.3.4 3.4 Server Failures	30
13.3.5 3.5 Semantic Errors	30
13.3.6 3.6 Security Errors	30
13.4 4. In what situations is gRPC a better choice than REST?	31
13.4.1 4.1 When Performance and Latency Are Critical	31
13.4.2 4.2 When You Need Bidirectional Streaming	31
13.4.3 4.3 When Strong Type Safety and Contract-First Development Are Required	31
13.4.4 4.4 When You Control Both Client and Server (Internal Microservices)	31
13.4.5 4.5 When You Need Built-in Advanced Features	31
13.4.6 4.6 When Bandwidth or Battery Is Limited (Mobile/Edge)	31
13.5 5. In what situations is REST a simpler or more appropriate choice?	32
13.5.1 5.1 When Building Public APIs	32
13.5.2 5.2 When Caching Is Critical for Performance	32
13.5.3 5.3 When Operations Are Naturally Resource-Oriented (CRUD)	32
13.5.4 5.4 When the Team Lacks RPC Experience	32
13.5.5 5.5 When You Need HATEOAS (Hypermedia as the Engine of Application State)	32
13.5.6 5.6 When Debugging and Human-Readability Matter	32
13.6 Summary: gRPC vs REST - Decision Matrix	33

14 Conclusion	33
14.1 Key Takeaways	33
14.2 Practical Guidelines for Choosing an RPC Technology	33
14.3 Final Remarks	34

1 Introduction

This document provides an **exhaustive, in-depth study** of Remote Procedure Call (RPC) as required for the second assignment of the *Fundamentals of Distributed Computing* course at the University of Tehran. It covers the definition of RPC, its historical origins, core concepts, detailed component roles, serialization and IDL intricacies, an exhaustive taxonomy of common errors, a comprehensive comparison with REST, and an evaluation of three major RPC technologies: gRPC, JSON-RPC, and XML-RPC (with additional references to Apache Thrift, Cap'n Proto, and SOAP). Furthermore, we discuss failure handling, idempotency, security, load balancing, protocol evolution, and advanced patterns. The document concludes with answers to five analytical questions, a use-case guide, a glossary, and extensive references.

2 Definition of RPC

Remote Procedure Call (RPC) is a **communication protocol** that allows a program to execute a procedure (function) on a remote server as if it were a local function call. The client calls a function, the request is marshaled (serialized) and sent over the network, the server executes the function, and the result is returned. The underlying network details — socket handling, data encoding, transport protocol, and error recovery — are hidden from the developer, drastically simplifying distributed programming.

2.1 Why “Remote” and “Procedure”?

The term “Remote” distinguishes it from a local procedure call (within the same process). “Procedure” refers to the traditional programming construct: a named block of code that takes parameters and returns a value. RPC extends this concept across machine boundaries.

2.2 Analogy: A Telephone Call

Think of RPC like making a phone call: you dial a number (the remote address), wait for the other side to pick up (connection), speak your request (parameters), listen to the reply (result), and hang up. The telephone network hides the details of signal routing, switching, and transmission. Similarly, RPC hides the network complexity.

2.3 Historical Background

RPC was introduced in the early 1980s by Bruce Jay Nelson as part of his PhD thesis at Carnegie Mellon University. The first widely used implementation was Sun RPC (also called ONC RPC) developed by Sun Microsystems in the mid-1980s, which became the basis for NFS (Network File System). Later, DCE/RPC (Distributed Computing Environment) was standardized by the Open Software Foundation and used in many Unix systems. In the 1990s, CORBA (Common Object Request Broker Architecture) emerged as a more complex object-oriented RPC system, but it was criticised for being too heavy. Modern RPC systems like gRPC (Google, 2015), Apache Thrift (Facebook, 2007), and Cap'n Proto (2013) build on these foundations, adding features like streaming, multiplexing, better serialization, and native HTTP/2 support. Each generation learned from the failures of its predecessors: CORBA's complexity, XML-RPC's verbosity, and Java RMI's language-lock-in.

2.4 Key Characteristics

- **Location transparency:** The caller does not need to know where the procedure is executed (which machine or process). This is a double-edged sword: it simplifies code but can hide the cost of network communication.
- **Synchronous by default:** The caller blocks until the reply arrives (though asynchronous/callback variants exist). Modern frameworks also support futures and promises.
- **Language-independent:** Stubs can be generated for many programming languages from a single IDL, enabling heterogeneous systems.

- **Interface contract:** The IDL defines the exact signature, parameter types, and return types, enabling strong type safety across languages. This contract is enforced at compile time for statically typed languages.
- **Transparent serialization:** Complex data structures (arrays, structs, maps) are automatically converted to a byte stream and reconstructed on the other side, freeing the developer from manual byte manipulation.

3 Core Idea: Calling a Remote Function

The core idea is to **extend the familiar local procedure call abstraction** to distributed environments. Instead of sending raw messages (e.g., ‘SEND "add" 5 3’ and then waiting for a reply using low-level sockets), the developer writes:

```
int result = remote\_add(5, 3);
```

The call appears identical to a local function call, but the actual computation occurs on another machine. This abstraction is achieved through **stubs** and **runtime libraries** that automatically handle:

- Marshaling arguments into a network message (e.g., Protocol Buffers, JSON, XML).
- Sending the message to the remote server using a transport protocol (TCP, UDP, HTTP/2).
- Waiting for the reply and unmarshaling the result.
- Handling transport-level errors (timeouts, retries, duplicate detection).

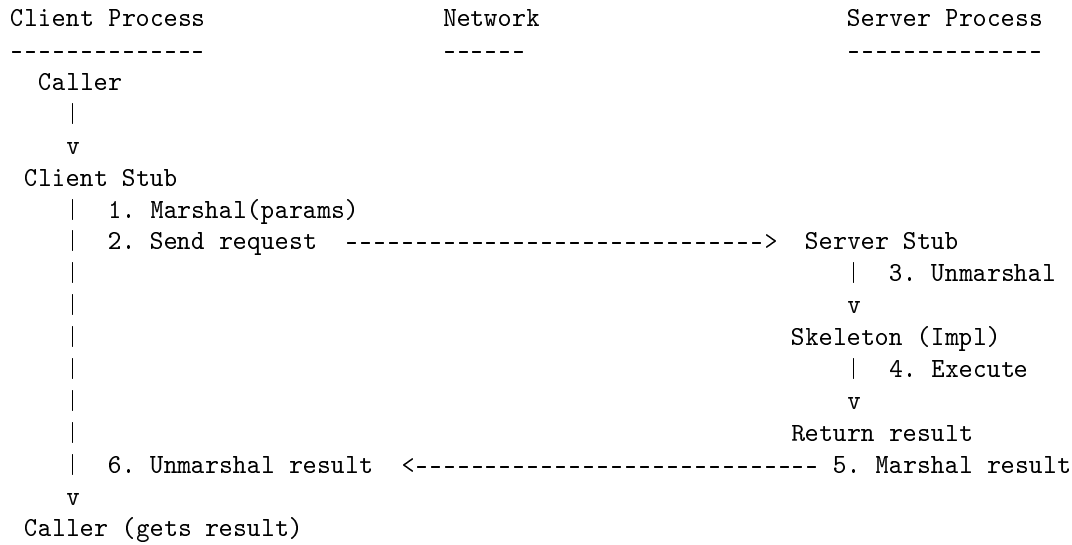
3.1 Detailed Call Flow (Step by Step)

We break down each step with more detail, including what happens at the network level.

1. The client application calls the client stub (proxy) with parameters. For example, ‘remote_add(5, 3)’.
2. The client stub marshals the parameters into a message (serialization). It uses a format like Protobuf, producing a byte array that might look like ‘\x08\x05\x10\x03’ (field numbers and values).
3. The client stub sends the message to the server via a transport protocol. This involves:
 - Resolving the server’s IP address (DNS).
 - Establishing a TCP connection (or reusing an existing one).
 - Sending the marshaled data, possibly with a length prefix.
 - Setting a timeout (deadline) for the response.
4. The server stub receives the message, unmarshals the parameters back into native data types (e.g., integers 5 and 3).
5. The server stub invokes the actual server implementation (the “skeleton”). For example, it calls ‘int add(int a, int b)’ with the unmarshaled values.
6. The server implementation returns a result (e.g., 8) to the server stub.
7. The server stub marshals the result into a byte stream and sends it back to the client over the same connection.
8. The client stub unmarshals the result (converting bytes back to an integer) and returns it to the caller, which receives ‘result = 8’.

3.2 Flow Diagram (Text Representation)

The following ASCII diagram shows the interaction:



4 Difference Between Local and Remote Invocation

The following table provides an exhaustive comparison of local and remote procedure calls across many dimensions. Understanding these differences is critical for designing robust distributed systems.

Table 1: Comprehensive comparison of local and remote procedure calls

Aspect	Local Invocation	Remote Invocation
Address space	Same memory space	Different processes, possibly different machines
Latency	1-10 ns	0.1-1000 ms (network RTT + processing)
Failure modes	Process crash, stack overflow, division by zero	Network partition, timeout, server crash, partial execution, serialization errors, security failures
Parameter passing	Pointers, references, values by copy	Only values (serialized); references cannot be transmitted directly
Concurrency	Threads share heap (synchronisation via mutexes)	Distributed concurrency, no shared memory (needs distributed locks or transactional systems)
Security	Typically none (process isolation)	Authentication (TLS, OAuth), encryption, access control, auditing
Debugging	Simple (step-through with gdb/IDE)	Complex (distributed traces, log aggregation, correlation IDs)
Versioning	Same binary (one deployment)	Multiple versions possible - requires schema evolution (e.g., Protobuf optional fields)
Cost	Free (negligible CPU cycles)	Non-negligible (network bandwidth, serialization CPU, memory buffers) - design must minimise RPC calls

Idempotency	Naturally idempotent (unless state changes)	Must be designed; network retries can cause duplicate execution (requires idempotent operations or deduplication)
Observability	Local logs, profilers	Requires distributed tracing (OpenTelemetry, Jaeger), metric aggregation (Prometheus)

4.1 Why the Differences Matter in Practice

- **Latency** means that calling a remote function in a loop can be devastating. Example: 1000 local calls might take 10 microseconds, but 1000 RPC calls could take 10 seconds.
- **Failure modes** force you to handle timeouts and retries. A local call never “times out”; an RPC can.
- **Parameter passing** means you cannot pass large arrays by reference; they are copied (serialized). A local call passes a pointer (cheap); an RPC copies the whole array (expensive).
- **Versioning** requires forward-compatible message designs. Without schema evolution, upgrading the server may break old clients.

This foundational understanding sets the stage for the rest of the report, where we explore how to build RPC systems that are resilient to these differences.

5 Role of Client Stub and Server Stub

Stubs are the **key components** that make RPC possible. They are automatically generated from an IDL and act as the “glue” between the application logic and the network. Without stubs, developers would have to manually write socket code, serialise data, and handle errors - a tedious and error-prone process.

5.1 Client Stub (Proxy)

The client stub is a local object that implements the same interface as the remote service. To the caller, it looks exactly like a local function. Behind the scenes, it performs five crucial tasks.

1. **Marshaling (Serialization):** Converts the procedure parameters (which may be complex nested structures) into a linear byte stream using a predefined format (e.g., Protobuf, JSON, XML). *Example:* A ‘LoginRequest’ with ‘username="alice"’ and ‘password="123"’ becomes a sequence of bytes. *Why it matters:* Different machines have different memory layouts; serialisation creates a platform-independent representation.
2. **Transport selection:** Opens a network connection to the remote server (or reuses a pooled connection), sets timeouts, and sends the byte stream. *Details:* It may use TCP, HTTP/2, or even UDP. It can also attach metadata (e.g., authentication tokens, trace IDs). *Why it matters:* Efficient connection reuse reduces latency and resource usage.
3. **Blocking/Async wait:** If the call is synchronous, the stub blocks the calling thread until a response arrives. If asynchronous, it registers a callback or returns a future/promise. *Example:* In Go, a synchronous gRPC call blocks until the reply or timeout. In Java, you can use ‘listenableFuture’. *Why it matters:* The caller’s expectation (blocking vs non-blocking) determines how concurrency is managed.
4. **Response unmarshaling:** Decodes the response byte stream back into the function’s return value and any out-parameters. *Example:* A ‘LoginResponse’ byte stream is turned back into a ‘success: true’ object. *Why it matters:* The caller receives a native data structure, not raw bytes.
5. **Error propagation:** Converts network errors, timeouts, or protocol errors into exceptions or error codes that the caller can handle. *Example:* A ‘DeadlineExceeded’ error from the transport layer becomes a gRPC status with code ‘DEADLINE_EXCEEDED’. *Why it matters:* The caller gets a consistent, meaningful error instead of a cryptic “connection reset”.

5.2 Server Stub (Skeleton)

The server stub runs on the remote machine and acts as a dispatcher. It listens for incoming requests, unpacks them, calls the actual business logic, and sends back the result.

1. **Listening:** Binds to a network port and accepts incoming RPC requests. May use HTTP/2, raw TCP, or custom transports. *Example:* A gRPC server listens on ‘:50051’ for HTTP/2 connections.
2. **Request receipt:** Reads the inbound byte stream from the transport. *Details:* The stub may read headers and trailers (e.g., in HTTP/2) to extract metadata.
3. **Unmarshaling:** Decodes the parameters from the byte stream into the local representation (e.g., a Protobuf message struct). *Example:* A byte stream becomes a ‘LoginRequest’ object with ‘username’ and ‘password’ fields.
4. **Invocation:** Calls the actual server implementation (the user-provided function) with the unmarshaled parameters. *Example:* ‘authImpl.Login(ctx, request)’ is invoked. The stub passes the context (which includes deadlines and cancellation).
5. **Result marshaling:** Encodes the return values and any out-parameters into a byte stream. *Example:* A ‘LoginResponsesuccess: true’ becomes a sequence of bytes.
6. **Response sending:** Transmits the marshaled result back to the client, possibly along with trailing metadata. *Example:* The byte stream is sent over the same HTTP/2 stream.
7. **Error handling:** If the user function panics or returns an error, the server stub marshals an error response (e.g., a gRPC status with code ‘INTERNAL’ or ‘INVALID_ARGUMENT’). *Example:* A missing required parameter results in a ‘INVALID_ARGUMENT’ error sent back to the client.

5.3 Code Generation and Tooling

In modern RPC systems (like gRPC and Thrift), these stubs are automatically generated by a compiler (e.g., ‘protoc’ for gRPC). The developer writes the IDL file, runs the generator, and the stubs are produced in the target language. This process:

- **Eliminates boilerplate:** No manual serialisation or networking code.
- **Ensures type safety:** The compiler checks that the client and server use the same interface.
- **Enables cross-language interoperability:** One IDL can generate stubs for Go, Java, Python, C++, etc.

6 Serialization and Deserialization

Serialization (marshaling) is the process of converting **data structures or objects** into a linear byte stream suitable for network transmission. Deserialization is the reverse process. The choice of serialization format affects performance, interoperability, security, and human readability.

6.1 Deep Dive into Serialization Formats

Format	Type	Characteristics
Protocol Buffers	Binary	Compact (3-10× smaller than JSON), fast, schema-based (‘.proto’), supports evolution, used by gRPC.
JSON	Text	Human-readable, language-agnostic, slower (parsing overhead), larger payload, no schema by default.
XML	Text	Very verbose, widely supported in legacy systems, schema (XSD) optional, slow.
MessagePack	Binary	Similar to JSON but binary, smaller than JSON, no schema, good for scripting.
Avro	Binary/JSON	Schema evolution, used in Apache Kafka, dynamic typing.
Thrift	Binary/JSON/Compact	Supports multiple serialization modes, used by Apache Thrift.
CBOR	Binary	Based on JSON, compact, used in COSE (IoT).

Table 2: Common Serialization Formats

6.2 Schema Evolution - Why It Matters

One of the most critical features for long-running distributed systems is **schema evolution**. In a microservice architecture, you cannot upgrade all clients and servers simultaneously. Different versions must coexist. Schema evolution allows you to change your data structures without breaking old clients.

6.2.1 Rules for Safe Evolution (Protobuf Example)

- **Field numbers are forever:** Each field has a unique number. Never change the meaning of a field number. When removing a field, mark the number as ‘reserved’ to prevent reuse.
- **Adding new fields is safe** as long as they are optional (or have a sensible default). Old clients ignore unknown fields.
- **Removing fields is safe** but you must ‘reserve’ the number. Old clients that still send that field will have it ignored by new servers (if they are compiled to ignore unknown fields).
- **Changing field types is dangerous:** Only changes that preserve wire format are allowed (e.g., ‘int32’ → ‘int64’ on some platforms, ‘string’ ↔ ‘bytes’). Most type changes break compatibility.
- **Renaming fields is allowed** because the wire format uses field numbers, not names.

6.2.2 Example of Evolution

Original ‘.proto’:

```
message User {  
  string name = 1;  
  int32 age = 2;  
}
```

New version (add email as optional):

```
message User {  
  string name = 1;  
  int32 age = 2;  
  optional string email = 3;  // new field  
}
```

Old clients still send only ‘name’ and ‘age’; new servers accept the message and set ‘email’ to empty. New clients may send all three; old servers ignore ‘email’ (if compiled with unknown field preservation). This works.

6.3 Performance Considerations

- **Binary formats** (Protobuf, Thrift, MessagePack) are significantly faster to parse and produce smaller messages than text formats. For high-throughput microservices, binary serialization is preferred.
- **Text formats** (JSON, XML) are slower, larger, but human-readable. They are suitable for debugging, administrative APIs, and when the client is a browser.
- **Zero-copy formats** (Cap’n Proto, FlatBuffers) allow direct memory access without parsing. They are extremely fast but more complex.

7 Interface Definition Language (IDL)

IDL is a **language-neutral description** of the service interface: the functions, parameter types, and return types. From the IDL file, tools automatically generate client and server stubs in various programming languages. This is the contract between the client and server.

7.1 Why IDL is Important

- **Contract enforcement:** Both sides agree on the exact interface. Changes are reviewed via the IDL.
- **Documentation:** The IDL serves as up-to-date documentation (unlike manually written docs that may drift).
- **Code generation:** Saves thousands of lines of boilerplate code.
- **Cross-language support:** A single IDL can generate stubs for Go, Java, Python, C++, Rust, etc.

7.2 Examples of IDLs

7.2.1 Protocol Buffers (‘.proto’)

- gRPC

```
syntax = "proto3";

package auth;

service AuthService {
  rpc Login(LoginRequest) returns (LoginResponse);
}

message LoginRequest {
  string username = 1;
  string password = 2;
}

message LoginResponse {
  bool success = 1;
}
```

Features: ‘message’, ‘service’, ‘rpc’, ‘enum’, ‘oneof’, ‘map’, ‘reserved’.

7.2.2 Thrift IDL

```
namespace go auth

struct LoginRequest {
  1: string username,
  2: string password
}

struct LoginResponse {
  1: bool success
}

service AuthService {
  LoginResponse Login(1: LoginRequest req)
}
```

Features: ‘struct’, ‘service’, ‘exception’, ‘typedef’, ‘enum’.

7.2.3 WSDL (XML)

- SOAP WSDL is an XML document describing a web service. It defines operations, messages, bindings, and endpoints. Very verbose but toolable. Example snippet:

```
<message name="LoginRequest">
  <part name="username" type="xsd:string"/>
  <part name="password" type="xsd:string"/>
</message>
<operation name="Login">
  <input message="tns:LoginRequest"/>
  <output message="tns:LoginResponse"/>
</operation>
```

7.2.4 OpenAPI (Swagger)

- REST (can describe RPC-like APIs)

```
openapi: 3.0.0
paths:
  /login:
    post:
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                username: {type: string}
                password: {type: string}
      responses:
        200:
          description: OK
          content:
            application/json:
              schema:
                type: object
                properties:
                  success: {type: boolean}
```

7.3 Example: Generating Stubs from ‘auth.proto’

Using the ‘protoc’ compiler:

```
protoc --go\out=. --go-grpc\out=. auth.proto
```

This generates:

- ‘auth.pb.go’ - Protobuf message types.
- ‘auth_grpc.pb.go’ - gRPC client and server interfaces.

The developer then implements the server interface:

```
type authServer struct {
    pb.UnimplementedAuthServiceServer
}

func (s *authServer) Login(ctx context.Context, req *pb.LoginRequest) (*pb.
    LoginResponse, error) {
    // business logic
    return &pb.LoginResponse{Success: true}, nil
}
```

The client stub is used as:

```
conn, _ := grpc.Dial(addr, grpc.WithInsecure())
client := pb.NewAuthServiceClient(conn)
resp, _ := client.Login(ctx, &pb.LoginRequest{Username: "alice", Password: "
    123"})
```

Notice how the generated stubs provide a clean, type-safe interface.

7.4 Best Practices for IDL Design

- **Use semantic field names:** ‘user_id’ not ‘fl’.
- **Reserve field numbers for future use:** Prevent reuse of deleted fields.
- **Keep messages small and focused:** Decompose large structures into smaller ones.
- **Use ‘oneof’ for mutually exclusive fields:** Avoids having to check multiple fields.
- **Document the IDL:** Comments in the IDL file become part of the generated documentation.
- **Version your IDL files:** Put them in version control; treat changes like code changes (code review, testing).

7.5 IDL vs. No IDL (e.g., JSON-RPC)

JSON-RPC has no mandatory IDL. You document the API manually. This is simpler for small projects but leads to:

- **Manual mismatches:** The client and server can drift without the compiler catching errors.
- **No type safety:** You must handle missing fields, wrong types at runtime.
- **Poor tooling:** No code generation, no stub generation, no compile-time checks.

For large, long-lived systems, an IDL is highly recommended. For quick scripts or internal tools, JSON-RPC’s simplicity may outweigh the benefits of an IDL.

8 Common RPC Errors - Exhaustive Taxonomy

RPC introduces failure modes that do not exist in local calls. Understanding them is crucial for building robust distributed systems. We classify errors into six categories, each with specific causes, examples, and mitigation strategies.

8.1 1. Network Errors

Network errors occur when the underlying communication channel fails. They are among the most common failures in distributed systems.

- **Connection refused:** The server process is not listening on the specified port. *Example:* The server has crashed or not yet started. *Mitigation:* Retry with exponential backoff, use health checks, and ensure the server is registered with service discovery.
- **Connection timeout:** The TCP handshake or data transfer times out due to network congestion, firewall, or overloaded server. *Example:* A misconfigured firewall drops SYN packets. *Mitigation:* Reduce the connection timeout, use circuit breakers, and ensure network policies allow the traffic.
- **DNS resolution failure:** Hostname cannot be resolved to an IP address. *Example:* The DNS server is unreachable or the hostname is misspelled. *Mitigation:* Use IP addresses for critical services, implement DNS caching, and have fallback endpoints.
- **Packet loss (UDP-based RPC):** Requests or replies are lost; no automatic retransmission. *Example:* A noisy network line corrupts UDP datagrams. *Mitigation:* Use TCP instead of UDP, or implement application-level acknowledgements and retransmission.
- **Network partition:** Complete loss of connectivity between client and server (e.g., cable cut, switch failure). *Example:* A data centre link is accidentally unplugged. *Mitigation:* Deploy redundant network paths, use multi-region replication, and design for eventual consistency.

8.2 2. Protocol Errors

Protocol errors arise when the client and server do not speak the same language or the message is corrupted.

- **Protocol version mismatch:** Client and server use different versions of the RPC protocol (e.g., gRPC over HTTP/2 vs. an older custom protocol). *Example:* Upgrading the server to a new major version without updating clients. *Mitigation:* Use version negotiation, backward-compatible protocol evolution, or separate endpoints for different versions.
- **Malformed message:** The byte stream does not conform to the expected format (e.g., missing required field, invalid length prefix, premature EOF). *Example:* A buggy client sends a truncated Protobuf message. *Mitigation:* Validate messages on arrival, return clear error codes (e.g., gRPC `INVALID_ARGUMENT`), and log the offending data.
- **Serialization error:** Type mismatch or unknown field in strict mode (e.g., JSON field missing a required property, or an integer where a string is expected). *Example:* The server expects a `'timestamp'` field but the client omits it. *Mitigation:* Use schema evolution (optional fields, defaults), and avoid strict mode in production.

8.3 3. Timeout Errors

A timeout occurs when the client does not receive a response within a specified deadline. Timeouts are a critical tool for preventing cascading failures.

- **Client timeout:** The client sets a deadline (e.g., 5 seconds). If the server does not reply within that time, the client raises a timeout error. However, the server may still execute the request after the deadline (orphaned request). *Example:* A database query takes 10 seconds, but the client times out after 2 seconds. The query completes later, wasting resources. *Mitigation:* Set deadlines based on the 99th percentile latency; use idempotent operations so orphaned requests are harmless; implement request cancellation propagation.
- **Server timeout:** The server itself may have a deadline and abort processing if the request takes too long. *Example:* A server uses a 30-second timeout for a file upload; if exceeded, it closes the connection. *Mitigation:* Coordinate deadlines between client and server (propagate the client's deadline via metadata).

8.4 4. Server Failures

Server failures range from crashes to overload conditions. They are unavoidable, but their impact can be minimised.

- **Crash before execution:** The server dies before any processing; the client never receives a response. *Example:* The server process is killed by an OOM killer. *Mitigation:* The client can safely retry because no state was changed. Use health checks to route away from unhealthy instances.
- **Crash during execution:** The server may have partially applied changes (e.g., updated a database but did not commit). At-most-once semantics require the client to know whether the operation succeeded. *Example:* A bank transfer partially deducts from account A but crashes before crediting B. *Mitigation:* Use idempotent operations, distributed transactions, or sagas.
- **Crash after execution but before reply:** The operation succeeded, but the client never receives the reply. Retrying would duplicate the operation (if non-idempotent). *Example:* An email is sent, but the server crashes before sending the HTTP 200 OK. *Mitigation:* Use idempotency keys, or design operations to be idempotent (e.g., ‘sendEmail’ with a unique ‘requestId’).
- **Load shedding / throttling:** The server rejects the request because it is overloaded (e.g., returns gRPC ‘RESOURCE_EXHAUSTED’, HTTP 429, or a custom “busy” error). *Example:* A server has a maximum concurrency of 100; the 101st request is rejected. *Mitigation:* Client-side backoff (exponential backoff with jitter), circuit breakers, and load balancing across many replicas.

8.5 5. Semantic Errors

Semantic errors are logical mistakes in how requests are handled, often due to assumptions about ordering, uniqueness, or side effects.

- **Idempotency violation:** A non-idempotent operation retried leads to duplicate records, double charges, etc. *Example:* A ‘POST /payment’ that does not check for duplicate requests will charge the user twice if the client retries due to a timeout. *Mitigation:* Design operations to be idempotent (e.g., use PUT with a resource ID), or use idempotency keys.
- **Ordering violation:** Requests may arrive out of order due to retries or network delays. *Example:* “Add item” arrives before “Create cart” because the latter was retried. *Mitigation:* Use sequence numbers, causal ordering, or design operations to be commutative (order-independent).
- **Partial failure (ambiguous outcome):** The client does not know whether the server executed the request. This is the hardest semantic error. *Example:* A ‘Transfer’ RPC times out; the server might have completed the transfer, or might not have. *Mitigation:* Use idempotency keys, transaction IDs, and design for “at-most-once” semantics with idempotent operations.

8.6 6. Security Errors

Security errors stem from insufficient protection of the RPC channel or credentials.

- **Authentication failed:** Invalid credentials, expired token, missing TLS client certificate. *Example:* An API key is revoked but the client still uses it. *Mitigation:* Implement token refresh, mutual TLS (mTLS), and return clear authentication errors.
- **Authorization denied:** The principal is authenticated but lacks permission to call the method. *Example:* A regular user tries to call an admin-only RPC. *Mitigation:* Use role-based access control (RBAC), and return ‘PERMISSION_DENIED’.
- **Encryption errors:** TLS handshake failure, certificate mismatch, unsupported cipher suite. *Example:* The server certificate has expired. *Mitigation:* Automate certificate renewal (e.g., cert-manager), use trusted CAs, and avoid insecure cipher suites.
- **Man-in-the-middle attack:** No encryption or compromised certificate allows an attacker to eavesdrop or modify traffic. *Example:* A public Wi-Fi hotspot injects malicious responses. *Mitigation:* Always use TLS in production, pin certificates for critical services, and validate server identity.

9 Comparison of RPC and REST

REST (Representational State Transfer) is an architectural style for designing networked applications, often using HTTP. The following table contrasts RPC with REST across many dimensions. Below the table, we provide a deep analysis of each criterion.

Criteria	RPC	REST
Abstraction	Procedure call (action/verb)	Resource manipulation (nouns)
HTTP methods	Usually POST (or GET for idempotent)	GET, POST, PUT, PATCH, DELETE, HEAD, OPTIONS
URI design	‘/procedureName’ or ‘/service/method’	‘/resources/id’ or ‘/resources’
Idempotency	Not guaranteed; depends on method implementation	GET, PUT, DELETE are idempotent by HTTP specification
Caching	No inherent support; custom headers	Built-in via ‘Cache-Control’, ‘ETag’, ‘Last-Modified’
Error handling	Application-specific error codes (e.g., ‘1001’, ‘1002’)	HTTP status codes (2xx, 3xx, 4xx, 5xx)
Tooling	Code generation from IDL (stubs)	OpenAPI/Swagger, but often manual
Streaming	Native in gRPC (bidirectional)	Requires WebSockets or Server-Sent Events (SSE)
Performance	Very high (binary, HTTP/2 multiplexing)	Medium (JSON/XML over HTTP/1.1)
Discoverability	Low (need IDL or documentation)	High (HATEOAS, hypermedia links)
Statefulness	Can be stateful (session cookies, connection affinity)	Stateless by design (each request contains all information)
Payload size	Small (binary)	Larger (text)
Human-readability	Poor (binary)	Excellent (JSON/XML)

Table 3: RPC vs REST - comprehensive comparison

9.1 Detailed Analysis of Each Criterion

9.1.1 Abstraction

RPC models the system as a set of procedures or methods. Clients call these procedures, passing arguments and receiving results. This is natural for action-oriented tasks (e.g., ‘computeTax’, ‘sendEmail’). REST models the system as a set of resources (nouns) and a fixed set of operations (HTTP verbs). This is natural for CRUD applications (e.g., ‘GET /customers/123’, ‘POST /orders’).

9.1.2 HTTP Methods

RPC typically uses POST for all actions (though some use GET for idempotent queries). REST uses the full spectrum of HTTP methods, each with well-defined semantics (GET is safe, PUT is idempotent, etc.). This makes REST APIs more self-descriptive.

9.1.3 Idempotency

In REST, GET, PUT, DELETE are idempotent by specification. In RPC, idempotency is not automatic; the developer must ensure that repeated calls have the same effect. For example, a ‘createAccount’ RPC is not idempotent unless it uses an idempotency key.

9.1.4 Caching

REST leverages HTTP caching headers, allowing intermediaries (CDNs, proxies) to cache responses. This can drastically reduce server load. RPC has no built-in caching mechanism; you must implement it manually (e.g., using Redis or a custom cache layer).

9.1.5 Error Handling

REST uses standard HTTP status codes (200, 400, 404, 500, etc.), which are well understood. RPC often uses application-specific error codes or gRPC's rich status codes (which are an improvement over plain numeric codes). However, gRPC codes are not as widely recognised outside the gRPC ecosystem.

9.1.6 Tooling

RPC frameworks like gRPC and Thrift generate client and server stubs from an IDL. This provides type safety and reduces boilerplate. REST APIs are often described with OpenAPI (Swagger), but the tooling is less integrated; you still write a lot of manual HTTP handling.

9.1.7 Streaming

RPC (specifically gRPC) natively supports bidirectional streaming over a single HTTP/2 connection. REST does not support streaming; to achieve similar effects, you need WebSockets or Server-Sent Events (SSE), which are separate protocols.

9.1.8 Performance

gRPC (binary + HTTP/2) is significantly faster than REST over HTTP/1.1 (text + per-request connections). For high-throughput systems, gRPC is the clear winner. However, for many applications, REST performance is “good enough”.

9.1.9 Discoverability

REST with HATEOAS (Hypermedia as the Engine of Application State) provides discoverable APIs: clients can navigate by following links. RPC APIs require external documentation (e.g., a ‘.proto’ file) or a service reflection mechanism.

9.1.10 Statefulness

REST is stateless by design: each request contains all the information needed to process it. This simplifies scaling and fault tolerance. RPC can be stateful (e.g., using session cookies or connection affinity), but that adds complexity and reduces scalability.

9.1.11 Human-readability

REST messages are usually JSON or XML, which are text-based and easy to inspect with ‘curl’ or browser dev tools. gRPC's binary Protobuf messages are not human-readable; you need tools like ‘grpcurl’ or ‘protoc’ to decode them.

9.2 When to Choose Which?

Based on the analysis above, here are concrete guidelines.

- **Choose RPC (especially gRPC) for:**
 - **Internal microservices** with high performance requirements (low latency, high throughput).
 - **Streaming** (real-time updates, chat, live data feeds).
 - **Polyglot environments** where strong type safety and code generation are beneficial.
 - **Mobile backends** where small message size saves battery and bandwidth.
 - **Systems that need deadlines, flow control, and advanced load balancing** out of the box.
- **Choose REST for:**
 - **Public APIs** that must be accessible to many different clients (browsers, mobile apps, third-party developers).
 - **CRUD-oriented applications** where resources map naturally to URLs.

- **When caching is critical** (HTTP caching reduces server load and latency).
- **When the team has deep HTTP expertise** and no RPC experience.
- **When you need HATEOAS** or hypermedia discoverability.
- **When debugging and human-readability are paramount** (e.g., during development or for administrative APIs).

In many modern systems, a hybrid approach is common: use gRPC for internal service-to-service communication, and expose a REST API (with Swagger) for external clients. This gives the best of both worlds.

10 Comparison of Popular RPC Technologies

We compare three widely used RPC technologies: **gRPC**, **JSON-RPC**, and **XML-RPC**. Additionally, we briefly mention Apache Thrift, Cap'n Proto, and SOAP. The goal is to give you not just a list of features, but a deep understanding of when and why each technology shines, along with their trade-offs.

10.1 gRPC - Deep Dive

gRPC is a modern, high-performance RPC framework originally developed by Google and now maintained by the Cloud Native Computing Foundation (CNCF). It is designed for low-latency, high-throughput distributed systems.

10.1.1 Protocol Buffers (Protobuf) - The Engine

- **Binary serialisation:** Unlike JSON or XML, Protobuf encodes data in a compact binary format. A typical message is 3-10× smaller than its JSON equivalent, saving network bandwidth and memory.
- **Schema evolution:** You can add new fields without breaking old clients. Field numbers (not names) identify fields, and unknown fields are ignored by default.
- **Code generation:** The ‘.proto’ file is compiled into client and server stubs in many languages. This eliminates manual marshaling and ensures type safety at compile time.

10.1.2 HTTP/2 - The Transport

- **Multiplexing:** A single TCP connection can carry many concurrent RPC calls. No need for multiple connections or connection pooling.
- **Bidirectional streaming:** Both client and server can send multiple messages asynchronously over the same stream - ideal for chat, live updates, or large file transfers.
- **Flow control:** Prevents a fast sender from overwhelming a slow receiver.

10.1.3 Performance and Benchmarks

- gRPC can be up to 10× faster than JSON over HTTP/1.1 (REST) for similar operations, according to public benchmarks.
- The combination of binary serialisation and HTTP/2 multiplexing reduces CPU usage and latency.
- For example, a gRPC call may take 1-2 ms while a REST call might take 10-20 ms for the same logic.

10.1.4 Built-in Production Features

- **Deadlines and cancellation:** Propagated via context; client can set a timeout, and the server honours it.
- **Load balancing:** Client-side load balancing (round-robin, pick first) and integration with service discovery (Consul, etcd, Kubernetes).
- **Authentication:** TLS/SSL, OAuth2, token - pluggable via interceptors.
- **Interceptors (middleware):** Easily add logging, metrics, retries, or distributed tracing.

10.1.5 Limitations of gRPC

- **Browser support:** Native browsers cannot speak gRPC directly; you need a proxy (gRPC-web) or use REST.
- **Human-readability:** Binary messages are not easy to inspect with ‘curl’ - you need tools like ‘grpcurl’.
- **Learning curve:** Understanding Protocol Buffers, HTTP/2, and the code generation workflow takes time.

10.2 JSON-RPC - Deep Dive

JSON-RPC is a lightweight, text-based RPC protocol that uses JSON for encoding. It is deliberately simple and easy to implement.

10.2.1 Message Format

- A request is a JSON object with fields: ‘jsonrpc’, ‘method’, ‘params’, and ‘id’.
- A response has ‘jsonrpc’, ‘result’ (or ‘error’), and ‘id’.
- Example: ‘{"jsonrpc":"2.0","method":"add","params":[3,5],"id":1}’.

10.2.2 Key Features

- **No mandatory IDL:** You document the API manually. Any language that can parse JSON can act as a client or server.
- **Notifications:** A request without an ‘id’ field does not expect a response - useful for fire-and-forget.
- **Batch requests:** You can send an array of requests in one HTTP call, reducing round trips.
- **Human-readable:** You can test with ‘curl’ and inspect logs easily.

10.2.3 Use Cases

- **Cryptocurrency clients:** Bitcoin Core and many altcoins use JSON-RPC as their main API.
- **Internal tools and scripts:** When you need a quick RPC without code generation.
- **Rapid prototyping:** Very low friction.

10.2.4 Limitations

- **No streaming:** Each request/response is separate. To emulate streaming, you need WebSockets or long polling.
- **No built-in schema evolution:** Missing fields are just missing; you must handle them manually.
- **Performance:** JSON parsing is slower than binary, and messages are larger.

10.3 XML-RPC - Deep Dive

XML-RPC is a legacy protocol from the late 1990s that uses XML for encoding. It was a precursor to SOAP.

10.3.1 Message Structure

- A request is an XML document: `<methodCall><methodName>add</methodName><params><param><value>`
- Data types: int, boolean, string, double, dateTime.iso8601, base64, struct, array.
- Faults: `<fault><value><struct><member><name>faultCode</name><value><int>4</int></value></me`

10.3.2 Why It Still Exists

- Many legacy systems (e.g., WordPress, some CMS, old enterprise software) still expose XML-RPC APIs.
- Almost every programming language has an XML-RPC library (often dating back 20 years).
- It is simple enough to implement from scratch if needed.

10.3.3 Limitations

- **Verbosity:** An XML message can be 10-20× larger than its JSON equivalent, and parsing XML is CPU-intensive.
- **No advanced features:** No streaming, no built-in authentication, no batching.
- **Obsolete:** Most new projects avoid XML-RPC in favour of JSON-RPC or gRPC.

10.4 Additional Technologies - Brief Overview

10.4.1 Apache Thrift

- **Developed by Facebook** (now Apache). Supports many languages (C++, Java, Python, Go, PHP, etc.).
- **Flexible transport:** You can choose TCP, HTTP, or even Unix sockets.
- **Multiple serialisation formats:** binary, compact, JSON.
- Used in large-scale systems like Cassandra, HBase, and Hadoop.
- Compared to gRPC, Thrift offers more transport flexibility but less HTTP/2 integration.

10.4.2 Cap'n Proto

- **Zero-copy serialisation:** The in-memory representation is the same as the wire format. No encoding/decoding step - just pointer arithmetic.
- **RPC with promise pipelining:** You can send requests before previous ones finish, similar to futures.
- **Extremely fast:** Benchmarks show it can be 100-1000× faster than Protobuf for large messages.
- **Trade-off:** Smaller ecosystem, fewer tools, and less adoption than gRPC.

10.4.3 SOAP (Simple Object Access Protocol)

- **XML-based enterprise protocol** with heavy specifications (WS-Security, WS-ReliableMessaging).
- **WSDL (Web Services Description Language):** An XML-based IDL.
- **Still widely used** in banking, government, and legacy enterprise systems.
- **Very heavyweight:** Complex tooling, slow, and difficult to debug.

10.4.4 Java RMI (Remote Method Invocation)

- **Java-only RPC** using object serialisation.
- No separate IDL - you use Java interfaces.
- Requires an RMI registry for service lookup.
- Prone to versioning issues (Java serialisation is fragile).
- Almost never used in new Java projects today (replaced by gRPC or REST).

Table 4: Detailed Comparison of gRPC, JSON-RPC, and XML-RPC

Criteria	gRPC	JSON-RPC	XML-RPC
Data format	Protocol Buffers (binary)	JSON (text)	XML (text)
IDL	Required (‘.proto’)	Optional	Optional
Multi-language support	Excellent (11+ languages)	Any language with JSON	Any language with XML
Performance	Very high	Medium	Low (very verbose)
Streaming	Yes (bidirectional)	No	No
Ease of implementation	Moderate (code generation)	Easy	Easy
Error handling	Rich status codes	Error object	Fault element
Typical use cases	Microservices, low-latency, real-time	Simple APIs, internal tools, crypto	Legacy systems
HTTP/2 support	Yes	Optional (usually HTTP/1.1)	Optional (usually HTTP/1.1)
Deadlines/Timeouts	Built-in	Must be implemented manually	Must be implemented manually
Flow control	Yes (HTTP/2)	No	No

10.5 Practical Selection Guidelines

- **Start with gRPC if:**
 - You control both client and server (internal microservices).
 - You need high performance, low latency, or streaming.
 - You want strong type safety and contract-first development.
- **Use JSON-RPC if:**
 - You need a simple RPC for a small script or internal tool.
 - Human-readability and easy debugging are paramount.
 - You do not want to learn an IDL or code generation.
- **Avoid XML-RPC for new projects** - use JSON-RPC or gRPC instead.
- **Consider Thrift if you need custom transports** (e.g., raw TCP) and many languages.
- **Consider Cap’n Proto only for extreme performance needs** (e.g., HFT, in-memory databases).

This detailed comparison should give you the confidence to choose the right RPC technology for your distributed system.

11 Use Case Scenarios - Detailed Recommendations

Choosing the right RPC technology (or REST) depends on many factors: performance requirements, team expertise, network environment, need for streaming, and long-term maintainability. The following table provides a high-level overview, but the subsections below explain each scenario in depth.

Use Case	Recommended Technology	Reason
High-performance microservices (low latency, high throughput)	gRPC	Binary protocol, HTTP/2 multiplexing, streaming, load balancing.
Browser-to-server API (public)	REST (or JSON-RPC)	Simple JSON, easy debugging, caching, wide browser support.
Real-time streaming (chat, logs, push notifications)	gRPC (bidirectional streaming)	Native streaming, low overhead, built-in flow control.
Simple script calling a remote function (internal)	JSON-RPC	Lightweight, human-readable, no code generation, easy to test with 'curl'.
Legacy enterprise integration	XML-RPC (or SOAP)	Existing infrastructure, interoperability with old systems.
Mobile client backend (bandwidth limited)	gRPC + Protobuf	Small message size, low battery usage, efficient.
IoT devices (constrained resources)	gRPC (with HTTP/2) or MQTT	Efficient, but MQTT may be better for extremely constrained devices.
Cross-language systems with custom transport	Apache Thrift	Supports many languages and serialisation formats, flexible transport.
Zero-copy, ultra-high performance	Cap'n Proto	No serialisation overhead, direct memory access.

Table 5: Technology Recommendations by Use Case

11.1 High-performance Microservices

When you have dozens or hundreds of small services that communicate frequently. Example: an e-commerce platform with product service, order service, payment service, etc.

- **Why gRPC?**
 - **Binary protocol** (Protobuf) messages are 3-10× smaller than JSON, reducing network bandwidth.
 - **HTTP/2 multiplexing** allows many concurrent requests over one TCP connection, eliminating head-of-line blocking.
 - **Client-side load balancing** spreads traffic evenly without a central proxy.
 - **Deadlines and cancellation** propagate automatically, enabling graceful degradation.
- **Real-world example:** Google's internal microservices use gRPC extensively. Netflix uses gRPC for inter-service communication in its video streaming backend.

11.2 Browser-to-server Public API

When your API is called directly from web browsers (JavaScript) or mobile apps. Example: a weather API, a social media API.

- **Why REST?**
 - Browsers speak HTTP/1.1 and JSON natively. gRPC requires a proxy (gRPC-web) and is less mature.

- **Caching** with ‘Cache-Control’ headers reduces server load and latency for read-heavy endpoints.
- **Idempotent methods** (GET, PUT, DELETE) align well with HTTP semantics.
- **Statelessness** simplifies scaling and fault tolerance.
- **Alternative JSON-RPC:** Some APIs (e.g., Bitcoin Core RPC) use JSON-RPC over HTTP. It is simple but lacks cache semantics.

11.3 Real-time Streaming

When data flows continuously from server to client or both directions. Example: live chat, stock ticker, multiplayer game state sync.

- **Why gRPC bidirectional streaming?**
 - One persistent HTTP/2 connection carries many messages in both directions.
 - **Low overhead** - no per-message connection establishment.
 - **Flow control** prevents a slow reader from being overwhelmed.
 - **Cancellation** allows clients to stop receiving messages without tearing down the connection.
- **Alternative:** WebSockets with custom RPC framing. This requires more manual work and lacks standardised semantics.

11.4 Simple Internal Script Calling a Remote Function

When you need to call a remote procedure from a short shell script or a lightweight automation tool. Example: a deployment script that calls a restart RPC on a server.

- **Why JSON-RPC?**
 - No IDL, no code generation - you can craft the request with ‘echo ’"jsonrpc":"2.0","method":"restart","params'.
 - Test with ‘curl’ - no special libraries.
 - Human-readable - easy to debug.
- **Note:** For production, you might still want gRPC for type safety. But for ad-hoc scripts, JSON-RPC is perfect.

11.5 Legacy Enterprise Integration

When you must communicate with systems built in the 1990s or early 2000s that only support XML-based protocols. Example: banking mainframes, old CMS systems.

- **Why XML-RPC or SOAP?**
 - These systems already have stable XML-RPC or SOAP endpoints.
 - Many languages have mature libraries (e.g., ‘xmlrpc’ in Python, ‘soap’ in Java).
 - WS-* standards (security, transactions) are only available in SOAP.
- **Caution:** Modernising these systems is costly. Use a gateway to translate XML-RPC to JSON/-gRPC for new services.

11.6 Mobile Client Backend

When the client is a mobile app running on limited battery and cellular data. Example: a news app, a ride-hailing app.

- **Why gRPC + Protobuf?**
 - Small message size reduces data usage (important for users with metered plans).
 - Faster parsing saves CPU cycles and extends battery life.
 - gRPC supports client-side streaming (e.g., uploading large files in chunks).
- **Google’s own mobile apps (e.g., YouTube, Gmail)** use gRPC internally.

11.7 IoT Devices (Constrained Resources)

When devices have very limited memory, CPU, and power. Example: temperature sensors, smart light bulbs.

- **Why MQTT or CoAP?**
 - MQTT is designed for low bandwidth and unreliable networks; it uses a publish-subscribe model and tiny packets.
 - gRPC (even with HTTP/2) still has overhead that may be too high for 8-bit microcontrollers.
- **If you must use RPC**, consider gRPC with Protocol Buffers on more powerful edge gateways, not on the sensors themselves.

11.8 Cross-language Systems with Custom Transport

When you need to support many programming languages and have exotic transport requirements (e.g., raw TCP, UDP, Unix sockets). Example: a distributed database with custom replication protocol.

- **Why Apache Thrift?**
 - Supports C++, Java, Python, Go, PHP, Ruby, Erlang, Haskell, etc.
 - You can choose the transport (framed, unframed, HTTP) and protocol (binary, compact, JSON) independently.
 - Thrift is battle-tested in systems like Cassandra, HBase, and Hadoop.
- **Note:** gRPC is also cross-language (11+ languages) but is tied to HTTP/2. Thrift gives you more transport flexibility.

11.9 Zero-copy, Ultra-high Performance

When every microsecond counts and you cannot afford serialisation overhead. Example: high-frequency trading, real-time analytics, in-memory databases.

- **Why Cap'n Proto?**
 - The serialisation format is also the in-memory layout. Reading a message requires zero parsing - just pointer arithmetic.
 - RPC with promise pipelining allows you to send requests before the previous ones finish.
 - Benchmark: Cap'n Proto can be 100-1000× faster than Protobuf for large messages.
- **Trade-off:** Less tooling, smaller ecosystem, and not as widely adopted as gRPC.

12 Advanced Topics in RPC

12.1 Idempotency and Exactly-Once Semantics

12.1.1 Definition and Importance

Idempotency means that performing an operation multiple times has the same effect as performing it once. In distributed systems, idempotency is essential because network failures can cause clients to retry requests without knowing whether the server executed them.

12.1.2 Examples of Idempotent Operations

- GET /user/123: Reading data never changes state → idempotent.
- PUT /user/123 {name:"Alice"}: Setting the same value multiple times yields the same final state → idempotent.
- DELETE /user/123: Deleting a non-existent user is harmless → idempotent.

12.1.3 Non-Idempotent Operations

- `POST /transfer`: Transferring \$10 from account A to B - doing it twice transfers \$20.
- `POST /increment`: Incrementing a counter - each call increases the value.

12.1.4 Mitigation Strategies

- **Idempotency keys**: Client generates a unique key for each request. Server stores the key and the outcome. If the same key arrives again, the server returns the cached response without re-executing.
- **Design for idempotency**: Use PUT instead of POST for updates, or make each operation reversible (e.g., 'transfer' uses a transaction ID, and repeating the same ID is ignored).

12.1.5 Exactly-Once Semantics

Achieving exactly-once is extremely hard and often overkill. It requires:

- A distributed consensus protocol (Paxos, Raft) to agree on which requests have been processed.
- Persistent storage of request outcomes.
- Transactions that span the network and the storage layer.

In practice, most systems use **at-most-once** (gRPC default) with idempotent operations, or **at-least-once** with deduplication.

12.2 Timeouts and Deadlines

12.2.1 Why Deadlines are Critical

Without a deadline, a client could wait forever for a response from a crashed or slow server. This leads to:

- Thread exhaustion (each stuck request consumes a thread).
- Resource leaks (open connections, memory buffers).
- Cascading failures (a downstream timeout can block upstream services).

12.2.2 How gRPC Implements Deadlines

In gRPC, you pass a 'context' with a deadline:

```
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
resp, err := client.Login(ctx, &req)
```

The deadline propagates through the entire call chain. If the server does not respond in time, the client receives a 'DeadlineExceeded' error.

12.2.3 Choosing a Deadline

- Too short → false positives (the server would have succeeded given more time).
- Too long → clients wait unnecessarily, increasing tail latency.
- Typical rule: measure the 99th percentile latency and set the deadline to 2-3× that value.

12.3 Security in RPC

12.3.1 Encryption (TLS/SSL)

All production RPC should use TLS to prevent eavesdropping and man-in-the-middle attacks. gRPC integrates with TLS via 'credentials.NewTLS()'.

12.3.2 Authentication and Authorisation

- **mTLS (Mutual TLS):** Both client and server present certificates. Common in service meshes (Istio, Linkerd).
- **OAuth2 / JWT tokens:** Stateless tokens that carry claims (e.g., user ID, roles). gRPC supports token-based authentication via interceptors.
- **API keys:** Simpler but less secure; used for internal services.

12.3.3 JSON-RPC and XML-RPC Security

These can be secured by running over HTTPS (HTTP + TLS). Authentication is usually implemented at the application level (e.g., a ‘token’ field in the JSON request).

12.3.4 Common Pitfalls

- Disabling certificate verification (‘insecure’ mode) in production - never do this.
- Not rotating certificates - set up automatic renewal (e.g., cert-manager in Kubernetes).
- Logging sensitive data - avoid logging passwords or tokens.

12.4 Load Balancing and Service Discovery

12.4.1 Why Load Balancing is Needed

When you have multiple instances of a service (for scalability or fault tolerance), clients should distribute requests evenly. Otherwise, a single instance becomes overloaded while others are idle.

12.4.2 Client-Side vs Proxy-Side Load Balancing

- **Client-side (gRPC):** The client knows all server addresses and picks one (round-robin, least loaded). Benefits: no extra hop, lower latency.
- **Proxy-side (REST, JSON-RPC):** A reverse proxy (Nginx, HAProxy) receives requests and distributes them. Simpler for REST but adds one network hop.

12.4.3 Service Discovery

Clients need a way to find the current set of server addresses. Solutions:

- **DNS:** Round-robin DNS returns multiple IPs. Simple but not instant on failure.
- **Consul / etcd / Zookeeper:** Centralised registries. Services register themselves, clients watch for changes.
- **Kubernetes endpoints:** The kube-proxy provides service discovery and load balancing.

gRPC has pluggable name resolvers that can integrate with Consul, etcd, or custom logic.

12.5 Protocol Evolution and Backward Compatibility

12.5.1 Why Schema Evolution Matters

In a distributed system, you cannot upgrade all clients and servers simultaneously. Different versions must coexist. A change to the message format must not break older versions.

12.5.2 Protobuf Evolution Rules

- **Adding a new field:** Safe if it is optional (or has a default). Old servers ignore unknown fields (by default).
- **Removing a field:** Mark it ‘reserved’ in the ‘.proto’ file to prevent reuse of the field number. Old clients may still send it; the server should ignore it.
- **Changing field type:** Not allowed except for compatible cases (e.g., ‘int32’ → ‘int64’ on some platforms).
- **Changing field name:** Allowed because the wire format uses field numbers, not names.

12.5.3 JSON-RPC and XML-RPC Evolution

These formats have no built-in schema. You must:

- Add fields that are optional (clients ignore missing fields).
- Never remove a field that old clients might still send; instead, ignore it.
- Use versioning in the URL (e.g., ‘/v1/rpc’, ‘/v2/rpc’) for breaking changes.

12.6 Observability and Distributed Tracing

12.6.1 The Three Pillars of Observability

1. **Metrics:** Counters, histograms, gauges. Example: number of RPC calls, latency percentiles, error rate.
2. **Tracing:** Follow a single request across multiple services. Example: OpenTelemetry + Jaeger.
3. **Logging:** Structured logs with correlation IDs. Example: JSON logs with ‘request_id’.

12.6.2 Correlation IDs

A unique ID generated at the entry point (e.g., by the web server) and passed through all subsequent RPC calls via metadata. This allows you to correlate logs from different services.

12.6.3 Tracing with gRPC

gRPC supports interceptors that inject and extract trace context. OpenTelemetry provides gRPC plugins:

```
import "go.opentelemetry.io/contrib/instrumentation/google.golang.org/grpc/otelgrpc"

conn, err := grpc.Dial(addr,
    grpc.WithUnaryInterceptor(otelgrpc.UnaryClientInterceptor()),
    grpc.WithStreamInterceptor(otelgrpc.StreamClientInterceptor()),
)
```

This automatically adds trace spans for each RPC call.

12.6.4 Manual Implementation for JSON-RPC / XML-RPC

For technologies without built-in observability, you can:

- Add a ‘request_id’ field to every request.
- Log the ID in both client and server logs.
- Measure latency manually by timing the round trip.

However, this is error-prone and less powerful than automatic instrumentation.

12.7 Conclusion of Advanced Topics

Understanding idempotency, deadlines, security, load balancing, schema evolution, and observability is essential for building robust distributed systems. While gRPC provides many of these features out of the box, even REST and JSON-RPC can be used effectively if you apply the same principles manually. The choice of technology should be guided by the specific requirements of your system, not by fashion.

13 Answers to Analytical Questions

13.1 1. Why does RPC make a remote service call appear like a local function call?

Remote Procedure Call achieves this illusion through the **client stub** (or proxy). The client stub is a piece of code automatically generated from an Interface Definition Language (IDL). It provides the exact same function signature (name, parameters, return type) as the remote procedure. Internally, the stub performs the following steps transparently:

1. **Marshaling (Serialization)**: Converts the function arguments into a byte stream that can travel over the network (e.g., using Protocol Buffers, JSON, or XML).
2. **Transport**: Opens a network connection (or reuses a pooled one) and sends the byte stream to the remote server.
3. **Blocking wait**: Suspends the calling thread until a response arrives or a timeout occurs.
4. **Unmarshaling**: Decodes the response byte stream back into the function's return value (and any out-parameters).
5. **Return**: Gives the result back to the caller, exactly like a local function would.

From the perspective of the calling code, all these steps are invisible. The developer writes:

```
// Local call
result := localAdd(3, 5)

// Remote call (appears identical)
result := remoteAdd(3, 5)
```

This abstraction is the very essence of RPC. It hides the complexity of distributed programming - socket handling, concurrency, serialization, error recovery - behind a familiar, local-looking interface. This dramatically reduces the cognitive load and allows developers to focus on business logic rather than low-level networking.

13.2 2. Why can this similarity be dangerous or misleading?

The abstraction is said to **leak** - it hides some complexities but not all, and the hidden parts can cause severe problems in production. The following table summarises the key *fallacies of distributed computing* as they apply to RPC.

Fallacy	Why it is false in RPC
The network is reliable	RPC calls can fail due to packet loss, router crashes, DNS failures, etc.
Latency is zero	Network round-trips take milliseconds to seconds; local calls take nanoseconds.
Bandwidth is infinite	Serialised payloads consume bandwidth; large messages can cause congestion.
The network is secure	RPC needs explicit encryption (TLS) and authentication; plaintext RPC is vulnerable.
Topology doesn't change	Servers can be added, removed, or fail; clients need service discovery.
There is only one administrator	Firewalls, proxies, and network policies can block RPC traffic.
Transport cost is zero	Connection establishment, TLS handshakes, and marshaling use CPU and memory.
The call is instantaneous	A remote call may block the caller for seconds, causing thread starvation.

Table 6: Fallacies of distributed computing applied to RPC

Consequences of assuming RPC is just like a local call:

- **Performance bottlenecks:** A developer might call an RPC inside a tight loop (e.g., ‘for i := 0; i < 1000; i++ rpc.GetUser(i) ‘). Each call adds at least one network round-trip, turning a microsecond operation into a multi-second disaster.
- **Hanging applications:** If a developer forgets to set a deadline (timeout), a slow or dead server can cause the client to wait indefinitely, exhausting threads and freezing the entire system.
- **Data corruption from partial failures:** The client may time out, but the server might have already executed the request (e.g., a money transfer). Without idempotency, retrying the call would double-charge the user.
- **Memory and CPU overhead:** Passing a large data structure (e.g., a 10 MB image) by reference in a local call is cheap. In RPC, the entire structure must be serialised and copied over the network, consuming bandwidth and memory.

This is known as the “**fallacy of RPC**” - the mistaken belief that remote calls are as reliable, fast, and cheap as local ones. Experienced distributed systems engineers learn to treat every RPC as a potential failure point and design accordingly (timeouts, retries, circuit breakers, idempotency).

13.3 3. What errors can occur in remote invocations that do not exist in local invocations?

The following expanded taxonomy categorises errors unique to RPC, along with their causes and typical mitigation strategies.

13.3.1 3.1 Network Errors

- **Connection refused:** The server process is not listening on the specified port (e.g., not started, crashed, or wrong port). Mitigation: retry with exponential backoff, use health checks.
- **Connection timeout:** TCP handshake or data transfer times out due to network congestion, firewall, or overloaded server. Mitigation: reduce timeout, use circuit breakers.
- **DNS resolution failure:** The hostname cannot be resolved to an IP address. Mitigation: use IP addresses or robust DNS caching.
- **Packet loss (UDP-based RPC):** Requests or replies are lost; no automatic retransmission. Mitigation: use TCP or implement application-level acknowledgements.
- **Network partition:** Complete loss of connectivity between client and server (e.g., cable cut, switch failure). Mitigation: use redundant network paths, deploy multiple replicas.

13.3.2 3.2 Protocol and Marshaling Errors

- **Protocol version mismatch:** Client and server use different versions of the RPC protocol (e.g., gRPC over HTTP/2 vs. HTTP/1.1). Mitigation: version negotiation or backward-compatible upgrades.
- **Malformed message:** The byte stream does not conform to the expected format (e.g., missing required field in a Protobuf message, invalid JSON). Mitigation: schema validation, graceful error handling.
- **Serialization error:** Type mismatch (e.g., client sends a string where server expects an integer), or unknown field in strict mode. Mitigation: use schema evolution (optional fields), tolerant reader pattern.

13.3.3 3.3 Timeout Errors

- **Client timeout:** The client sets a deadline (e.g., 5 seconds). If the server does not reply within that time, the client raises a timeout error. However, the server may still execute the request after the timeout (orphaned request). Mitigation: use idempotent operations and client-side retry budgets.
- **Server timeout:** The server itself may have a deadline and abort processing if the request takes too long. Mitigation: tune deadlines based on typical processing times.

13.3.4 3.4 Server Failures

- **Crash before execution:** Server dies before any processing; client can safely retry because no side effects have occurred.
- **Crash during execution:** Server may have partially applied changes (e.g., updated a database but not sent the reply). At-most-once semantics require the client to know whether the operation was applied. Mitigation: use idempotent operations and transaction IDs.
- **Crash after execution but before reply:** The operation succeeded, but the client never receives the response. Retrying would duplicate the operation (if non-idempotent). Mitigation: idempotency keys, deduplication logs.
- **Load shedding / throttling:** Server rejects the request because it is overloaded (e.g., returns gRPC status 'RESOURCE_EXHAUSTED' or HTTP 429). Mitigation: client-side backoff, circuit breakers.

13.3.5 3.5 Semantic Errors

- **Idempotency violation:** A non-idempotent operation retried leads to duplicate records, double billing, etc. Mitigation: design operations to be idempotent or use request IDs.
- **Ordering violation:** Requests may arrive out of order due to retries or network delays. Mitigation: sequence numbers, causal ordering protocols.
- **Partial failure (ambiguous outcome):** The client does not know whether the server executed the request. This is the hardest error to handle. Mitigation: use idempotency and design for “at-most-once” semantics.

13.3.6 3.6 Security Errors

- **Authentication failed:** Invalid credentials, expired token, missing client certificate. Mitigation: implement token refresh, mutual TLS.
- **Authorization denied:** Authenticated but the principal lacks permission to call the method. Mitigation: fine-grained access control, audit logs.
- **Encryption errors:** TLS handshake failure, certificate mismatch, unsupported cipher suite. Mitigation: use trusted certificates, automated renewal.

- **Man-in-the-middle attack:** No encryption or compromised certificate. Mitigation: enforce TLS with certificate pinning in critical environments.

13.4 4. In what situations is gRPC a better choice than REST?

gRPC is a high-performance RPC framework that uses Protocol Buffers (binary) and HTTP/2. It is not a universal replacement for REST, but it excels in specific scenarios.

13.4.1 4.1 When Performance and Latency Are Critical

- Binary serialisation (Protobuf) produces messages 3-10× smaller than JSON, and parsing is much faster.
- HTTP/2 multiplexing allows many concurrent RPC calls over a single TCP connection, reducing connection overhead.
- Use case: **High-frequency trading systems** where microseconds matter; **real-time analytics** pipelines.

13.4.2 4.2 When You Need Bidirectional Streaming

- gRPC supports client-side, server-side, and bidirectional streaming natively over a single HTTP/2 connection.
- Use case: **Chat applications, live video/audio streaming, real-time dashboard updates, long-running job progress reports.**

13.4.3 4.3 When Strong Type Safety and Contract-First Development Are Required

- The ‘.proto’ file serves as an unambiguous contract between client and server.
- Code generation produces type-safe stubs, eliminating manual marshaling and reducing integration bugs.
- Use case: **Large microservice ecosystems** with multiple teams; **polyglot environments** (Go, Java, Python, C++).

13.4.4 4.4 When You Control Both Client and Server (Internal Microservices)

- No need for HTTP caching or browser compatibility.
- You can optimise the network and deploy service discovery.
- Use case: **Internal backend services** in a data centre or Kubernetes cluster.

13.4.5 4.5 When You Need Built-in Advanced Features

- **Deadlines (timeouts):** Propagated automatically via context.
- **Load balancing:** Client-side round-robin, pick first, etc., with service discovery integration.
- **Interceptors:** Easy to add logging, metrics, retries, and authentication.
- Use case: **Production microservices** requiring resilience and observability.

13.4.6 4.6 When Bandwidth or Battery Is Limited (Mobile/Edge)

- Small message size reduces cellular data usage.
- Faster processing saves battery life.
- Use case: **Mobile app backends, IoT devices.**

13.5 5. In what situations is REST a simpler or more appropriate choice?

REST is not just “HTTP with JSON”. It is an architectural style that uses resources and HTTP verbs. It remains superior for many scenarios.

13.5.1 5.1 When Building Public APIs

- REST uses standard HTTP methods (GET, POST, PUT, DELETE) and status codes, which are familiar to all web developers.
- Clients can be written in any language that speaks HTTP - no special libraries or code generation required.
- Use case: **Public APIs for mobile apps, web frontends, third-party integrations** (e.g., Stripe, GitHub, Twitter).

13.5.2 5.2 When Caching Is Critical for Performance

- REST leverages HTTP caching headers (‘Cache-Control’, ‘ETag’, ‘Last-Modified’). Responses can be cached by CDNs, reverse proxies, and browsers.
- This reduces server load and latency for read-heavy workloads.
- Example: A ‘GET /posts/42’ response can be cached for 5 minutes, avoiding repeated RPC calls.

13.5.3 5.3 When Operations Are Naturally Resource-Oriented (CRUD)

- REST maps naturally to Create (POST), Read (GET), Update (PUT/PATCH), Delete (DELETE) on resources.
- Example: A blog system has resources like ‘/posts’, ‘/users’, ‘/comments’. The uniform interface simplifies design.
- RPC would require methods like ‘createPost()’, ‘getPost()’, ‘updatePost()’, etc. - more verbose and less discoverable.

13.5.4 5.4 When the Team Lacks RPC Experience

- REST/HTTP is well understood; there is no need to learn Protocol Buffers, gRPC error codes, or streaming APIs.
- Tools like ‘curl’, browser dev tools, and Postman work out of the box.
- Use case: **Small teams building simple web services.**

13.5.5 5.5 When You Need HATEOAS (Hypermedia as the Engine of Application State)

- REST can include hyperlinks in responses, allowing clients to discover actions dynamically.
- Example: A response to ‘GET /order/123’ may contain links like ‘"cancel": "/order/123/cancel"’. The client does not need hard-coded URL construction.
- This decouples clients from server URL structure.

13.5.6 5.6 When Debugging and Human-Readability Matter

- JSON is text-based and easy to inspect, log, and understand.
- Developers can reproduce bugs using ‘curl’ or a browser without installing special tools.
- gRPC’s binary format (Protobuf) is not human-readable; debugging requires tools like ‘grpcurl’.

13.6 Summary: gRPC vs REST - Decision Matrix

Requirement	gRPC	REST
Low latency / high throughput	✓ Excellent	× Higher overhead
Bidirectional streaming	✓ Native	× Requires WebSockets
Strong type safety	✓ (Protobuf)	× (OpenAPI optional)
Public API / broad client compatibility	× (needs gRPC-web)	✓ Native
HTTP caching	× Not inherent	✓ Built-in
Human-readable messages	× Binary	✓ JSON
Code generation	✓ Automatic	× Manual (or OpenAPI tools)
Service discovery	✓ Pluggable	× Usually manual (or DNS)
Browser support	× Requires gRPC-web	✓ Native

Table 7: Decision matrix for choosing between gRPC and REST

In practice, many systems use a hybrid approach: **gRPC for internal microservices** (performance, type safety, streaming) and **REST for public APIs** (caching, browser compatibility, simplicity). Understanding the strengths and weaknesses of each is essential for modern distributed systems design.

14 Conclusion

Remote Procedure Call (RPC) is one of the most influential abstractions in distributed computing. It allows developers to invoke functions on remote servers as if they were local, hiding the complexity of network communication, serialisation, and concurrency. However, as we have seen throughout this report, this abstraction is not perfect. The so-called *fallacy of RPC* reminds us that remote calls are fundamentally different from local ones: they have higher latency, they can fail in many new ways (network partitions, timeouts, partial failures), and they require careful design regarding idempotency, deadlines, and error handling.

14.1 Key Takeaways

- **RPC is not a silver bullet.** While it simplifies code for internal microservices, it can lead to hidden performance bottlenecks and resilience issues if used naively.
- **Choice of technology matters.** gRPC excels in high-throughput, low-latency, streaming scenarios; JSON-RPC is ideal for lightweight, human-debuggable APIs; XML-RPC (and SOAP) are legacy choices but still found in enterprise environments.
- **Security and observability must be built in.** Modern RPC systems (especially gRPC) provide built-in TLS, authentication, deadlines, and distributed tracing support - features that are essential for production systems.
- **Schema evolution is a lifesaver.** Using an IDL with binary serialisation (Protobuf, Thrift) allows services to evolve independently without breaking clients, a critical requirement for long-running distributed systems.
- **REST remains relevant.** For public APIs, CRUD applications, and when caching is paramount, REST is often simpler and more appropriate than RPC.

14.2 Practical Guidelines for Choosing an RPC Technology

Based on the analysis in this report, here is a decision flowchart in text form:

1. Do you need streaming (bidirectional or server push)?
 - Yes -> gRPC (or WebSockets + custom RPC)
 - No -> Continue

2. Do you control both client and server (internal services)?
 - Yes -> gRPC (strong typing, high performance)
 - No -> Continue
3. Is human-readability and ease of debugging critical?
 - Yes -> JSON-RPC (or REST)
 - No -> gRPC or Thrift
4. Are you integrating with legacy systems that only speak XML?
 - Yes -> XML-RPC or SOAP
 - No -> JSON-RPC or gRPC

14.3 Final Remarks

This report has provided a comprehensive, theoretically grounded and practically oriented study of RPC. We have covered definitions, historical context, detailed component roles (stubs, serialisation, IDL), an exhaustive taxonomy of errors, a point-by-point comparison with REST, and an in-depth evaluation of three major RPC technologies (gRPC, JSON-RPC, XML-RPC) with references to others (Thrift, Cap'n Proto, SOAP). We have also answered the five analytical questions with depth and clarity.

Understanding RPC is essential for any distributed systems engineer. This knowledge enables you to design resilient, high-performance, and maintainable services. The assignment's practical part (building a multi-VM system with gRPC, HTTP file serving, and Pub/Sub alerts) complements this theoretical study, giving you hands-on experience with the concepts discussed.

Glossary

Core RPC Terms

- **RPC (Remote Procedure Call)**: A protocol that allows a program to execute a procedure on a remote server as if it were a local function call. Hides network details.
- **Client Stub**: A local proxy object generated from an IDL. It marshals parameters, sends the request, waits for the reply, and unmarshals the result.
- **Server Stub (Skeleton)**: A generated dispatcher on the server side. It receives the request, unmarshals parameters, calls the actual implementation, and sends back the result.
- **IDL (Interface Definition Language)**: A language-neutral specification of the service's methods, parameters, and return types. Used to generate stubs.
- **Marshaling (Serialization)**: Converting data structures into a byte stream suitable for network transmission.
- **Unmarshaling (Deserialization)**: Reconstructing data structures from a byte stream.

RPC Technologies

- **gRPC**: High-performance RPC framework by Google. Uses Protocol Buffers and HTTP/2. Supports bidirectional streaming, deadlines, load balancing, and pluggable authentication.
- **Protocol Buffers (Protobuf)**: Binary serialisation format with a schema (`.proto`). Compact, fast, and supports schema evolution.
- **JSON-RPC**: Lightweight, text-based RPC using JSON. No mandatory IDL. Simple to implement and debug. Used in Bitcoin Core, Ethereum clients, and many internal tools.
- **XML-RPC**: Legacy RPC using XML. Very verbose, but widely available in older systems. Now largely replaced by JSON-RPC.
- **Apache Thrift**: RPC framework from Facebook. Supports multiple serialisation formats (binary, JSON, compact) and transports (TCP, HTTP). Generates stubs for many languages.

- **SOAP:** XML-based enterprise RPC protocol. Heavyweight, supports WS-* standards (security, reliability). Still used in banking and government.
- **Cap'n Proto:** Zero-copy serialisation and RPC. Data is directly usable from the wire - no encoding step. Extremely fast, but less adopted than gRPC.

Error and Reliability Concepts

- **Idempotent:** An operation that can be executed multiple times without changing the result beyond the first application. Examples: 'GET /user/123', 'DELETE /item/456'. Non-idempotent: 'POST /transfer'.
- **Deadline (Timeout):** The maximum time a client waits for an RPC response before giving up. Prevents indefinite hanging.
- **Partial Failure:** A situation where the server executes the request but the client never receives the reply (e.g., network drop after reply). The client cannot distinguish this from the server crashing before execution.
- **Exactly-once semantics:** The guarantee that an RPC is executed exactly one time, even in the presence of retries and failures. Extremely hard to achieve; usually requires idempotency keys or distributed transactions.
- **At-most-once semantics:** The call is executed either zero or one time. Used by most RPC frameworks (e.g., gRPC default).
- **At-least-once semantics:** The call may be executed one or more times. Suitable for idempotent operations.

Advanced Concepts

- **Streaming:** Continuous flow of messages between client and server. Can be client-side, server-side, or bidirectional (full duplex). Used in chat, live data feeds, file uploads.
- **Load balancing:** Distributing RPC requests across multiple server instances to improve throughput and availability. gRPC supports client-side load balancing (round-robin, pick first) and integrates with service discovery (Consul, etcd, Kubernetes).
- **Service discovery:** The mechanism by which a client finds the network address of a server (e.g., DNS, Consul, etcd).
- **Distributed tracing:** Tracking a request as it propagates through multiple services. Uses correlation IDs and span contexts. OpenTelemetry is the standard.
- **Schema evolution:** The ability to change a message definition (add, remove, or modify fields) without breaking existing clients. Protobuf and Avro support this; JSON-RPC does not.
- **Interceptor (Middleware):** A function that intercepts RPC calls on client or server side, allowing logging, authentication, metrics, retries, etc. gRPC provides interceptors.

Performance and Overhead

- **Binary vs Text formats:** Binary formats (Protobuf, MessagePack) produce smaller messages and parse faster, but are not human-readable. Text formats (JSON, XML) are larger and slower, but easier to debug.
- **Multiplexing:** HTTP/2 allows multiple RPC calls to share a single TCP connection, reducing connection overhead. gRPC uses this extensively.
- **Zero-copy:** A technique where data is sent from memory directly to the network without intermediate copies. Cap'n Proto achieves this by using a serialisation format that is also the in-memory representation.

References

Academic Papers

1. Birrell, A. D., & Nelson, B. J. (1984). Implementing remote procedure calls. *ACM Transactions on Computer Systems*, 2(1), 39-59. *The original RPC paper; defines the core concepts of stubs, marshaling, and transparency.*
2. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine. *Defines REST; explains why RPC is not considered a uniform interface and contrasts with resource-oriented design.*
3. Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3rd ed.). Pearson. *Chapter 4 (Communication) provides a comprehensive overview of RPC and its failure semantics.*

Official Documentation and Specifications

1. gRPC Documentation. <https://grpc.io/docs/> Includes tutorials, API references, and best practices for gRPC in multiple languages.
2. Protocol Buffers Language Guide. <https://protobuf.dev/programming-guides/proto3/> Specification of '.proto' syntax, field numbers, schema evolution rules.
3. JSON-RPC Specification (Version 2.0). <https://www.jsonrpc.org/specification> Official specification, including request/response formats, error objects, batch calls, and notifications.
4. XML-RPC Specification. <https://xmlrpc.com/spec.md> Original specification, describing data types and the '<methodCall>' / '<methodResponse>' structure.
5. Apache Thrift Documentation. <https://thrift.apache.org/docs/> IDL reference, transport and protocol layers, and code generation.
6. Cap'n Proto RPC Specification. <https://capnproto.org/rpc.html> Explains zero-copy serialisation, promise pipelining, and the RPC protocol.
7. WSDL 1.1 - Web Services Description Language. W3C Note, 2001. <https://www.w3.org/TR/wsdl> Describes how SOAP services are defined; includes operations, messages, and bindings.
8. OpenTelemetry Documentation. <https://opentelemetry.io/docs/> Standard for distributed tracing and metrics; includes instrumentation for gRPC.

Online Resources and Tutorials

1. "gRPC vs REST: Understanding the Differences" - Cloudflare Blog. <https://www.cloudflare.com/learning/performance/grpc-vs-rest/>
2. "RPC Idempotency: Why It Matters and How to Implement It" - Stripe Blog. <https://stripe.com/blog/idempotency>
3. "Deadlines and Cancellation in gRPC" - gRPC official documentation.
4. "Comparing Serialization Formats: Protobuf, JSON, MessagePack, and Avro" - Various benchmarks.

Course Materials

1. Dr. Mohammadreza Shourniya, *Fundamentals of Distributed Computing*, University of Tehran, Spring 2026. Course slides and lecture notes.
2. Assignment specification (provided by the course). Contains detailed requirements for the RPC study, multi-VM implementation, and Pub/Sub memory alerts.

Software Tools Used in the Assignment

1. Go programming language (version 1.23+). <https://go.dev/>
2. gRPC for Go. <https://pkg.go.dev/google.golang.org/grpc>
3. Protocol Buffers compiler ('protoc'). <https://github.com/protocolbuffers/protobuf>
4. VirtualBox (for running VMs). <https://www.virtualbox.org/>
5. Ubuntu 22.04 LTS (guest OS).